PARTNER DOCS · V1

Know the moment your customers claim

Liffio tells your system when a customer claims the perk you gave them, so you never send a second link to someone who already has it.

Webhooks `PUSH`
We POST to your server the moment a claim changes.

Status API `PULL`
You ask about one customer, any time.



HOW IT WORKS

From link to claim in five steps

The status changes as your customer moves through the claim. You only need to act on the last one.

1 **You send a link**
A signed link with your customer's ref, or a code.
`not_claimed`

2 **They open it**
We hold the perk for them for 30 minutes.
`pending`

3 **They sign up**
They log in to Liffio and connect Instagram.
`pending`

4 **Perk applied**
Their free plan starts right away.
`claimed`

5 **We tell you**
A `claim.claimed` webhook. Stop sending links. Later payments send `claim.paid`.
`claim.claimed`

CLAIM LINKS · SIGNED

Create a claim link for each customer

A signed link is a short token (a JWT) that says which customer it's for. You make it on your server with your **link signing secret**, which your Liffio contact gives you once. It's separate from the webhook secret.

The link

`URL` `https://liffio.com/claim/your-partner-slug#t=<token>`

The token goes after `#t=`, not `?t=`. The part after `#` never reaches any server or log, so the token stays private.

What goes in the token

`partner` `string` required
Your partner slug, exactly as in your claim URL.

`ref` `string · max 200` required
Your own id for this customer, for example your user id. Use the same ref for the same customer every time. It comes back to you in webhooks and the status API.

`iat` `unix seconds` required
When you made the link.

`exp` `unix seconds` required
When the link stops working. No later than your program's link lifetime after `iat` (Liffio tells you, for example 7 days).

`offer` `string` optional
Which of your offers this customer gets, for example `starter-2m`. Liffio gives you the list. Leave it out to use your default offer. A name that isn't on your list makes the link invalid.

`source` `string · max 64` optional
Where the link came from, for example `newsletter-nov`. Lowercase letters, numbers, `-` and `_`. Shows up in your reports. It doesn't change the offer.

Sign with **HS256**. Any other fields in the token are ignored.

Rules

1. **Sign on your server only.** Never put the secret in a web page, an app or an email template.
2. **No personal data in `ref`.** No emails or phone numbers: the ref is readable inside the link. Use an internal id.
3. **Check before you send.** Call the status API and only send a link when the status is `not_claimed`.
4. **One perk per ref.** A new link for a ref that already claimed only shows your customer "already claimed".

What your customer sees

Claim link · Node.js

```node
const jwt = require("jsonwebtoken");

function claimLink(ref) {
  const token = jwt.sign(
    { partner: "your-partner-slug", ref },
    process.env.LIFFIO_LINK_SECRET,
    { algorithm: "HS256", expiresIn: "7d" } // iat is added for you
  );
  return `https://liffio.com/claim/your-partner-slug#t=${token}`;
}

claimLink("123456789");
```

Claim link · Python

```python
import os, time, jwt  # pip install PyJWT

def claim_link(ref: str) -> str:
    now = int(time.time())
    token = jwt.encode(
        {"partner": "your-partner-slug", "ref": ref,
         "iat": now, "exp": now + 7 * 86400},
        os.environ["LIFFIO_LINK_SECRET"],
        algorithm="HS256",
    )
    return f"https://liffio.com/claim/your-partner-slug#t={token}"

claim_link("123456789")
```

Claim link · PHP

```php
use Firebase\JWT\JWT; // composer require firebase/php-jwt

function claimLink(string $ref): string {
    $now = time();
    $token = JWT::encode([
        'partner' => 'your-partner-slug',
        'ref'     => $ref,
        'iat'     => $now,
        'exp'     => $now + 7 * 86400,
    ], getenv('LIFFIO_LINK_SECRET'), 'HS256');
    return 'https://liffio.com/claim/your-partner-slug#t=' . $token;
}

claimLink('123456789');
```

OK	The perk page, held for them for 30 minutes while they sign up.
Bad	"This link isn't valid": wrong secret, wrong `partner`, or a missing field.
Old	"This link has expired": past `exp`, or `exp` is further from `iat` than your link lifetime.
Used	"Already claimed": this ref already has the perk.

Using codes too? You can use signed links, unique codes and a shared code together. Liffio turns on the ones you need. For codes, share `liffio.com/claim/your-partner-slug#code=LF-XXXX-XXXX` so the code fills in by itself, or let customers type it at `liffio.com/redeem`. That page takes any partner's code and switches to your offer once it finds the code. Codes need no signing.

Inside the token (decoded)

```
// header
{ "alg": "HS256", "typ": "JWT" }

// payload
{
  "partner": "your-partner-slug",
  "ref": "123456789",
  "iat": 1793952000,
  "exp": 1794556800,
  "offer": "starter-2m",
  "source": "newsletter-nov"
}
```

exp is 7 days after iat in this example

STATUSES

Five statuses, one simple rule

Webhooks and the status API use the same words. Only `not_claimed` means it's OK to send a link.

not_claimed	pending	claimed	blocked	revoked
Never claimed, or a 30 minute hold ran out.	On the 30 minute hold, or the customer is signing up.	The perk was applied, even after the free time ends.	Their Instagram account already used a Liffio perk.	Liffio cancelled this claim.
✓ OK to send a link	‖ Wait, check later	☐ Stop sending links	☐ Stop sending links	☐ Stop, ask Liffio

WEBHOOKS · PUSH

We POST to you when a claim changes

Give your Liffio contact an https URL, and we tell you about claims and payments. You get a signing secret (`whsec_...`) once, to check every request really came from us.

Events

Pick one to see its payload.

POST	`claim.claimed`	The perk was applied
POST	`claim.blocked`	Their Instagram account already used a Liffio perk
POST	`claim.revoked`	Liffio cancelled the claim

Request headers

```
POST https://your-server.example/liffio-webhook
Content-Type: application/json
User-Agent: Liffio-Webhooks/1.0
Liffio-Event-Id: evt_8f3k2a9d
Liffio-Signature: t=1793963201,v1=5f2b9c...
```

POST **claim.paid**

The customer paid. Includes your earning if you get revenue share

POST **claim.refunded**

That payment was refunded. Its earning is taken back

POST **ping**

A test from Liffio. `data` is `null`

No events are sent for a link being opened or a hold running out. Use the status API for those.

> **Revenue share.** `claim.paid` is sent for every payment from a customer you brought. `payment.earning` is your cut, or `null` when that payment doesn't earn (no revenue share, or the first payment came after your window). Earnings sit `on_hold` for 20 days, then become `available`. A refund sends `claim.refunded` and the earning becomes `clawed_back`.

What we do with your reply

Any 2xx within 10 s	→	**Delivered**
4xx, 5xx, timeout, server down	→	Retry later
A redirect (3xx)	→	Retry. We don't follow redirects

The reply body is ignored. A plain `200 OK` is enough.

Your side

1. **Check the signature** and reject anything that fails.
2. **Skip duplicates.** Retries reuse the same `Liffio-Event-Id`.
3. **Save the status** against `data.ref` (or `data.code`).
4. **Reply 200 fast** and do slow work afterwards.

> Events can arrive out of order. Trust `data.status`, not the order they arrive in.

Payload · claim.claimed

```
{
  "id": "evt_8f3k2a9d",
  "type": "claim.claimed",
  "created_at": "2026-11-06T10:00:01Z",
  "data": {
    "ref": "123456789",
    "code": null,
    "status": "claimed",
    "claimed_at": "2026-11-06T10:00:00Z",
    "plan": "starter",
    "access_ends_at": "2026-12-06T10:00:00Z",
    "pending_until": null
  }
}
```

Payload · claim.blocked

```
{
  "id": "evt_2k9d7c1x",
  "type": "claim.blocked",
  "created_at": "2026-11-06T10:00:01Z",
  "data": {
    "ref": "987654321",
    "code": null,
    "status": "blocked",
    "claimed_at": null,
    "plan": null,
    "access_ends_at": null,
    "pending_until": null
  }
}
```

Payload · claim.revoked

```
{
  "id": "evt_7h3m9q2p",
  "type": "claim.revoked",
  "created_at": "2026-11-07T09:15:00Z",
  "data": {
    "ref": "123456789",
    "code": null,
    "status": "revoked",
    "claimed_at": null,
    "plan": null,
    "access_ends_at": null,
    "pending_until": null
  }
}
```

Payload · claim.paid


```
{
  "id": "evt_3p8w1n6v",
  "type": "claim.paid",
  "created_at": "2026-12-04T08:12:09Z",
  "data": {
    "ref": "123456789",
    "code": null,
    "status": "claimed",
    "claimed_at": "2026-11-06T10:00:00Z",
    "plan": "starter",
    "access_ends_at": "2026-12-06T10:00:00Z",
    "pending_until": null,
    "payment": {
      "number": 1,
      "paid_at": "2026-12-04T08:12:05Z",
      "earning": {
        "amount": 149.7,
        "currency": "INR",
        "status": "on_hold",
        "available_at": "2026-12-24T08:12:05Z"
      }
    }
  }
}
```


Payload · claim.refunded


```
{
  "id": "evt_9r2k5t8x",
  "type": "claim.refunded",
  "created_at": "2026-12-09T11:30:44Z",
  "data": {
    "ref": "123456789",
    "code": null,
    "status": "claimed",
    "claimed_at": "2026-11-06T10:00:00Z",
    "plan": "starter",
    "access_ends_at": "2026-12-06T10:00:00Z",
    "pending_until": null,
    "payment": {
      "number": 1,
      "paid_at": "2026-12-04T08:12:05Z",
      "earning": {
        "amount": 149.7,
        "currency": "INR",
        "status": "clawed_back",
        "available_at": null
      }
    }
  }
}
```


Payload · ping


```
{
  "id": "evt_745e63017931",
  "type": "ping",
  "created_at": "2026-11-06T13:45:35Z",
  "data": null
}
```

If you don't reply 2xx, we keep trying for about 33 hours

Delay after the first try. Not to scale.



Same event id every time
So you can safely skip ones you've seen.

Turned off after 3 days
With no success and at least 20 failed tries.

Back on when fixed
Liffio turns it on again once your URL works.

SECURITY

Check every webhook is really from Liffio

`v1` is an HMAC-SHA256 of `<t>.<raw body>` with your signing secret, in hex. Use the raw body, before parsing JSON.

1. **Read `t` and every `v1`** from the `Liffio-Signature` header.
2. **Reject old requests.** If `t` is more than 5 minutes from now, drop it.
3. **Compute the HMAC** of `t`, a dot, and the raw body.
4. **Compare safely** with a constant-time compare. Any matching `v1` passes.

⚠ When Liffio rotates your secret, the header carries two `v1=` values for 24 hours. The code here accepts either, so nothing breaks.

Signature check · Node.js

```
const crypto = require("crypto");

function isFromLiffio(rawBody, header, secret) {
  const parts = header.split(",");
  const t = Number((parts.find((p) => p.startsWith("t=")) ||
"").slice(2));
  // reject anything older than 5 minutes
  if (!t || Math.abs(Date.now() / 1000 - t) > 300) return
false;

  const expected = crypto
    .createHmac("sha256", secret)
    .update(`${t}.${rawBody}`)
    .digest("hex");

  return parts
    .filter((p) => p.startsWith("v1="))
    .map((p) => p.slice(3))
    .some((s) =>
      s.length === expected.length &&
      crypto.timingSafeEqual(Buffer.from(s),
Buffer.from(expected))
    );
}
```

Signature check · Python

```python
import hashlib, hmac, time

def is_from_liffio(raw_body: bytes, header: str, secret: str)
→ bool:
    parts = header.split(",")
    t = next((p[2:] for p in parts if p.startswith("t=")),
None)
    # reject anything older than 5 minutes
    if not t or abs(time.time() - int(t)) > 300:
        return False

    expected = hmac.new(
        secret.encode(), f"{t}.".encode() + raw_body,
hashlib.sha256
    ).hexdigest()

    return any(
        hmac.compare_digest(p[3:], expected)
        for p in parts if p.startswith("v1=")
    )
```

STATUS API · PULL

Ask about one customer, any time

Call it from your server just before you send a link, or to catch a webhook you missed. Your API key (`lf_partner_...`) comes from your Liffio contact.

`GET` `https://api.liffio.com/partner/v1/claims/{ref}`

Headers

`Authorization` `string` required

`Bearer lf_partner_...`. Server side only, never in a web page or app.

Path

`ref` `string · max 200` required

Your customer reference, URL-encoded (a space becomes `%20`). A ref we've never seen returns `not_claimed`, not a 404.

Responses

200 Always the same six fields. Fields that don't apply are `null`.

400 `{"error":"invalid_ref"}` The ref is empty or too long.

401 `{"error":"unauthorized"}` Key missing, wrong or revoked.

429 `{"error":"rate_limited"}` Over 60 a minute. Wait the seconds in `Retry-After`.

Request · cURL

```bash
curl https://api.liffio.com/partner/v1/claims/123456789 \
  -H "Authorization: Bearer $LIFFIO_API_KEY"
```

Request · Node.js

```javascript
const ref = "123456789";
const res = await fetch(

`https://api.liffio.com/partner/v1/claims/${encodeURIComponent
(ref)}`,
  { headers: { Authorization: `Bearer
${process.env.LIFFIO_API_KEY}` } }
);
const claim = await res.json();

if (claim.status === "not_claimed") {
  // OK to send this customer a link
}
```

Using codes? Look up by code

`GET` `https://api.liffio.com/partner/v1/codes/{code}`

Same key, same limit, same six fields, with `code` in place of `ref`, plus `code_state`: `active`, `expired` or `voided`. Works for one-time codes. A shared code is used by many customers, so it returns `400 {"error":"shared_code"}`. A code that isn't yours returns `404 {"error":"unknown_code"}`.

Request · Python

```python
import os, requests
from urllib.parse import quote

ref = "123456789"
res = requests.get(
    f"https://api.liffio.com/partner/v1/claims/{quote(ref, safe='')}",
    headers={"Authorization": f"Bearer {os.environ['LIFFIO_API_KEY']}"},
    timeout=10,
)
claim = res.json()

if claim["status"] == "not_claimed":
    # OK to send this customer a link
    ...
```

200 response · status claimed

```json
{
  "ref": "123456789",
  "status": "claimed",
  "claimed_at": "2026-11-06T10:00:00Z",
  "plan": "starter",
  "access_ends_at": "2026-12-06T10:00:00Z",
  "pending_until": null
}
```

200 response · status pending

```json
{
  "ref": "123456789",
  "status": "pending",
  "claimed_at": null,
  "plan": null,
  "access_ends_at": null,
  "pending_until": "2026-11-06T10:30:00Z"
}
```

200 response · status not_claimed

```json
{
  "ref": "123456789",
  "status": "not_claimed",
  "claimed_at": null,
  "plan": null,
  "access_ends_at": null,
  "pending_until": null
}
```

200 response · status blocked

```
{
  "ref": "123456789",
  "status": "blocked",
  "claimed_at": null,
  "plan": null,
  "access_ends_at": null,
  "pending_until": null
}
```

200 response · status revoked

```
{
  "ref": "123456789",
  "status": "revoked",
  "claimed_at": null,
  "plan": null,
  "access_ends_at": null,
  "pending_until": null
}
```

GOOD TO KNOW

Limits and rules

60 /min
Status API requests per key. Headers show what's left.

10 s
Time your server has to reply to a webhook.

7 tries
Per webhook, over about 33 hours.

UTC
Every time ends in Z. 10:00Z is 3:30 PM in India.

- **Use both**
  Webhooks keep your records current. Check the status API right before sending, in case one was missed.
- **One perk per customer**
  A re-sent link never gives a second perk. It only shows your customer "already claimed".
- **No customer data**
  We never send emails, names or Instagram handles. Only the ref or code, status, plan and times.
- **Keys shown once**
  If a key or secret leaks, ask Liffio to revoke or rotate it. To test, ask Liffio to press Send test.